

TP 1 Base de données PostgreSQL avancée

Conception exploitation et préparation d'une base de connaissances

Contexte

Ce travail individuel prolonge l'atelier PostgreSQL et JSONB. Vous concevrez une base de données qui répond à un besoin métier, puis vous démontrerez son intégrité, ses fonctions avancées, sa performance et son utilisation à partir d'une application. Vous préparerez aussi un export documenté qui pourra servir plus tard à une activité de recherche assistée par intelligence artificielle.

Votre remise doit être reproductible. Il faut pouvoir créer la base, charger les données, exécuter les tests et lancer l'application en suivant uniquement votre fichier README.

Objectifs d'apprentissage

- Concevoir un **schéma relationnel** normalisé à partir de besoins fonctionnels.
- Combiner données relationnelles et JSONB lorsque chaque représentation est justifiée.
- Implanter des contraintes, des index, des vues, des routines et un déclencheur dans PostgreSQL.
- Analyser des requêtes avec **EXPLAIN ANALYZE** et mesurer l'effet d'un index.
- Exécuter des opérations paramétrées et une transaction depuis une application cliente.
- Produire un export traçable, propre et exploitable dans une future base de connaissances.

Modalités

- Travail individuel
- Pondération 20 %
- Durée estimée ~ 10h
- Date de remise 16 octobre 2026.
- Application client en C# avec Npgsql ou en Python avec psycopg.

Choix du scénario

Choisissez un scénario similaire à ceux-ci ou faites approuver un autre domaine. Votre scénario doit comporter des règles métier qui justifient au moins huit entités et plusieurs relations.

- **Gestion sportive** : équipes, joueurs, matchs, statistiques, saisons et classements.
- **Commerce en ligne** : produits, catégories, clients, commandes, paiements, expéditions et évaluations.
- **Réservation de services** : clients, prestataires, services, disponibilités, réservations, paiements et avis.
- **Gestion culturelle** : œuvres, créateurs, événements, lieux, billets, membres et réservations.
- **Suivi d'équipements** : équipements, capteurs, mesures, entretiens, techniciens, sites et alertes.

Travail demandé

Ce que le système fait

Analyse et conception

Que est-ce qu'une règle métier?

- Décrire le besoin, les utilisateurs visés et cinq règles métier vérifiables.
- Produire un diagramme entité relation comportant au moins huit entités, leurs cardinalités et les attributs essentiels.
- Transformer le modèle conceptuel en schéma relationnel et démontrer qu'il respecte la troisième forme normale. Toute dénormalisation doit être expliquée.
- Fournir un dictionnaire de données indiquant le type, l'obligation, la valeur par défaut et la règle de validation de chaque attribut important.
- Identifier les données qui conviennent au type JSONB et justifier ce choix par leur variabilité ou leur structure imbriquée.

Implémentation relationnelle et JSONB

- Créer toutes les tables avec des noms cohérents, des clés primaires et étrangères, des contraintes NOT NULL, UNIQUE, CHECK et DEFAULT pertinentes.
- Ajouter au moins trois index justifiés par des requêtes réelles. Les index créés automatiquement par les clés primaires ne comptent pas.
- Intégrer au moins une colonne JSONB contenant un objet imbriqué ou un tableau, ainsi qu'une contrainte validant une propriété essentielle.
- Créer un index GIN adapté à l'une de vos requêtes JSONB.
- Charger au moins 250 enregistrements cohérents. Une table centrale doit contenir au moins 100 lignes afin de permettre une première analyse de performance.
- Utiliser des données fictives. Les noms, courriels, adresses et identifiants ne doivent correspondre à aucune personne réelle.

Fonctions avancées de PostgreSQL

- Créer une vue utile à l'application ou au rapport.
- Créer une vue matérialisée destinée à un calcul ou à une synthèse, puis fournir la commande qui la rafraîchit.
- Créer une fonction PL/pgSQL qui reçoit au moins un paramètre et retourne une valeur ou un ensemble de lignes.
- Créer une procédure PL/pgSQL qui réalise une opération métier comportant plusieurs étapes.
- Créer une table d'audit et un déclencheur qui conserve l'opération, la date et les valeurs utiles après une modification importante.
- Créer un rôle de lecture et un rôle de modification, puis accorder uniquement les privilèges nécessaires.
- Démontrer une transaction avec COMMIT et une autre avec ROLLBACK. Expliquez la règle métier protégée par la transaction.

Requêtes et performance

Présentez dix requêtes distinctes qui répondent à des questions de votre scénario. Une même requête peut couvrir plusieurs notions, mais chaque notion ci-dessous doit apparaître clairement.

- une jointure impliquant au moins trois tables;
- une agrégation avec GROUP BY et HAVING;
- une sous requête corrélée ou une expression EXISTS;
- une expression de table commune avec WITH;
- une fonction de fenêtre avec OVER;
- une opération ensembliste avec UNION, INTERSECT ou EXCEPT;
- un appel à votre vue, votre vue matérialisée ou votre fonction;
- une requête d'écriture paramétrée exécutée par l'application;
- deux requêtes JSONB, dont une recherche par contenance avec @> et une manipulation de structure imbriquée.

Analyse des plans d'exécution

- Choisir deux requêtes dont la performance peut être améliorée par un index.
- Capturer EXPLAIN ANALYZE avant et après la création de l'index.
- Comparer le type de parcours, le coût estimé, le nombre de lignes et le temps mesuré.
- Expliquer honnêtement le résultat. PostgreSQL peut préférer un parcours séquentiel sur une petite table même si un index existe.

Application client

Créez une application console en C# ou en Python. Elle doit utiliser des variables d'environnement pour la connexion et des requêtes paramétrées pour toutes les valeurs saisies.

- afficher une liste provenant d'une jointure ou d'une vue;
- rechercher selon au moins deux critères fournis par l'utilisateur;
- ajouter une donnée en respectant les contraintes;
- modifier ou supprimer une donnée dans une transaction;
- appeler votre fonction ou votre procédure;
- gérer les erreurs sans afficher le mot de passe ni la chaîne de connexion complète.

Tests et reproductibilité

- Fournir un script principal qui exécute les scripts SQL dans le bon ordre et s'arrête à la première erreur.
- Fournir au moins six tests automatisés ou assertions SQL couvrant les contraintes, une routine, le déclencheur et les principaux résultats.
- Documenter l'installation, les variables d'environnement, la création de la base, l'exécution des tests et le lancement de l'application.
- Ne jamais inclure de mot de passe, de fichier .env réel, de dossiers bin, obj, .venv ou de cache dans la remise.

Préparation de la base de connaissances

Produisez un export CSV, JSON ou JSONL contenant au moins 50 unités documentaires utiles à votre scénario. Chaque ligne ou objet doit pouvoir être compris sans consulter directement les tables sources.

- un identifiant stable `source_id`;
- un contenu textuel ou une représentation lisible nommée `content`;
- des métadonnées pertinentes, par exemple le type, la date, la catégorie ou la source;
- aucun secret ni renseignement personnel réel;
- une requête ou un script reproductible qui régénère l'export à partir de PostgreSQL.

Ajoutez une fiche de données d'une page qui décrit la provenance, la période couverte, les champs, les valeurs manquantes, les limites et les utilisations prévues de l'export.

Usage de l'intelligence artificielle

L'IA générative est autorisée comme outil d'aide. Vous demeurez responsable de toutes les décisions de conception, de chaque requête et du code remis.

- Consignez dans IA.md les outils utilisés, les principaux prompts et les portions de la remise qu'ils ont influencées.
- Décrivez au moins deux vérifications effectuées et une correction apportée à une proposition de l'IA.
- Ne transmettez jamais de mot de passe, de donnée personnelle réelle ni de document confidentiel à un assistant.
- Ne remettez aucun code que vous êtes incapable d'expliquer, de tester ou de modifier.

Je peux vous demander une démonstration et une modification en direct.

L'incapacité à expliquer un élément de la remise sera prise en compte dans l'évaluation.

Livrables

Déposez dans Moodle une seule archive nommée **TP1_prenom_nom.zip**. Elle doit respecter la structure suivante.

- rapport.pdf avec le diagramme, les justifications, le dictionnaire de données, les résultats, les plans d'exécution et la fiche de données;
- README.md avec toutes les commandes nécessaires;
- IA.md avec la déclaration d'utilisation de l'IA;
- sql avec des scripts numérotés dans leur ordre d'exécution;
- app avec le projet C# ou Python et ses dépendances déclarées;
- export avec le fichier CSV, JSON ou JSONL et le script qui le génère;
- diagramme avec le fichier source modifiable et une version PNG ou PDF.

Grille d'évaluation

Critère	Indicateurs	Points
Analyse et conception	Besoins, règles métier, modèle d'au moins huit entités, normalisation et dictionnaire de données	15
Implémentation	Schéma exécutable, contraintes, données réalistes, JSONB et index GIN	20
Fonctions avancées	Vue, vue matérialisée, fonction, procédure, déclencheur, audit, rôles et transactions	20
Requêtes et performance	Dix requêtes exactes, couverture des notions et comparaison de deux plans d'exécution	20
Application cliente	Connexion sécurisée, opérations paramétrées, transaction, routine et gestion des erreurs	10
Base de connaissances	Export d'au moins 50 unités, métadonnées, reproductibilité et fiche de données	10
Qualité de la remise	Rapport clair, README reproductible, tests, structure de l'archive et déclaration de l'IA	5

Vérification avant la remise

- ☐ Tous les scripts s'exécutent dans l'ordre à partir d'une base vide.
- ☐ Les données respectent les contraintes et les règles métier.
- ☐ Les plans avant et après index sont fournis et commentés.
- ☐ L'application démarre selon les instructions du README.
- ☐ Les tests réussissent et couvrent les objets avancés demandés.
- ☐ L'export peut être régénéré et ne contient aucune donnée sensible.
- ☐ L'archive ne contient aucun mot de passe ni dossier généré.
- ☐ Le fichier IA.md décrit honnêtement l'aide reçue.

Ce travail prépare le projet final en équipe. Conservez vos scripts, vos données fictives et votre documentation afin de pouvoir les réutiliser et les améliorer.